

Computer Networks

Lecture 9

Shmulik Kaplan

Today

- Transport layer
  - TCP – Connection Management
  - TCP – Flow Control
  - TCP – Congestion Control
  - TCP - Summary
- Network Layer
  - Recap – Review and Introduction
  - Routing – Link State (Dijkstra)
  - Routing – Distance Vector (Bellman Ford)
    - Basic algorithm
    - Cost change
    - Poisoned reverse
    - Real life solution
  - Comparing & Summary

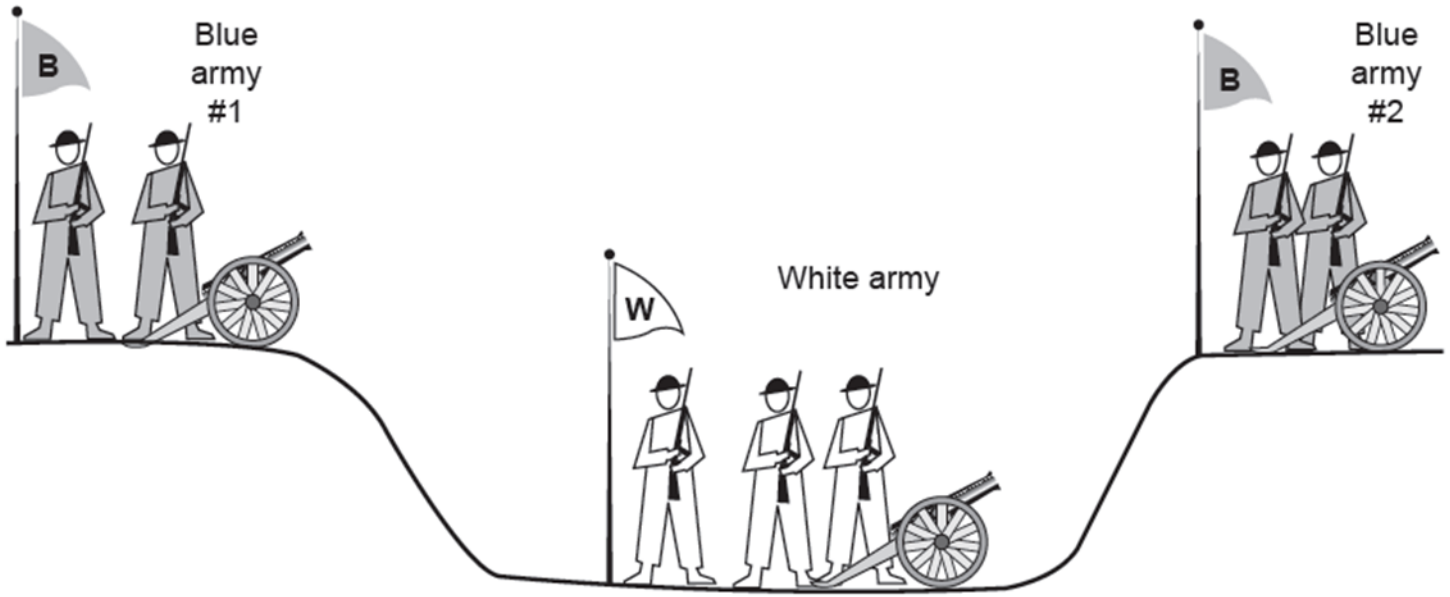
Today

- Transport layer
  - TCP – Connection Management
  - TCP – Flow Control
  - TCP – Congestion Control
  - TCP - Summary
- Network Layer
  - Recap – Review and Introduction
  - Routing – Link State (Dijkstra)
  - Routing – Distance Vector (Bellman Ford)
    - Basic algorithm
    - Cost change
    - Poisoned reverse
    - Real life solution
  - Comparing & Summary

The coordinated attack problem

בעיית ההתקפה המתואמת

(Closing a TCP connection)



TCP Connection Management (cont.)

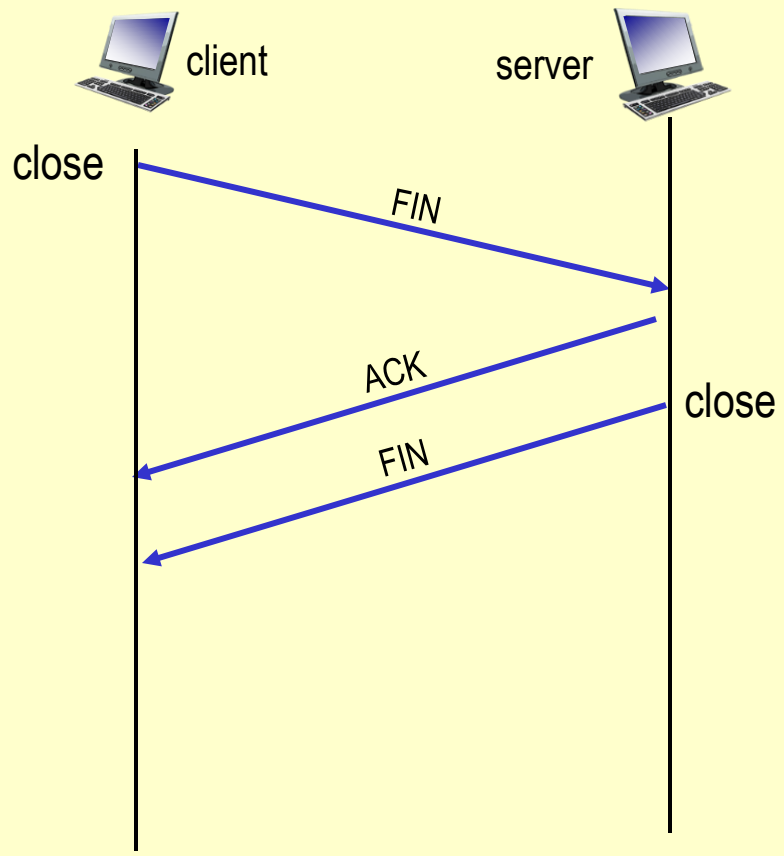
Closing a connection:

client closes socket:

clientSocket.close();

Step 1: client end system sends TCP FIN control segment to server

Step 2: server receives FIN, replies with ACK. Closes connection, sends FIN.

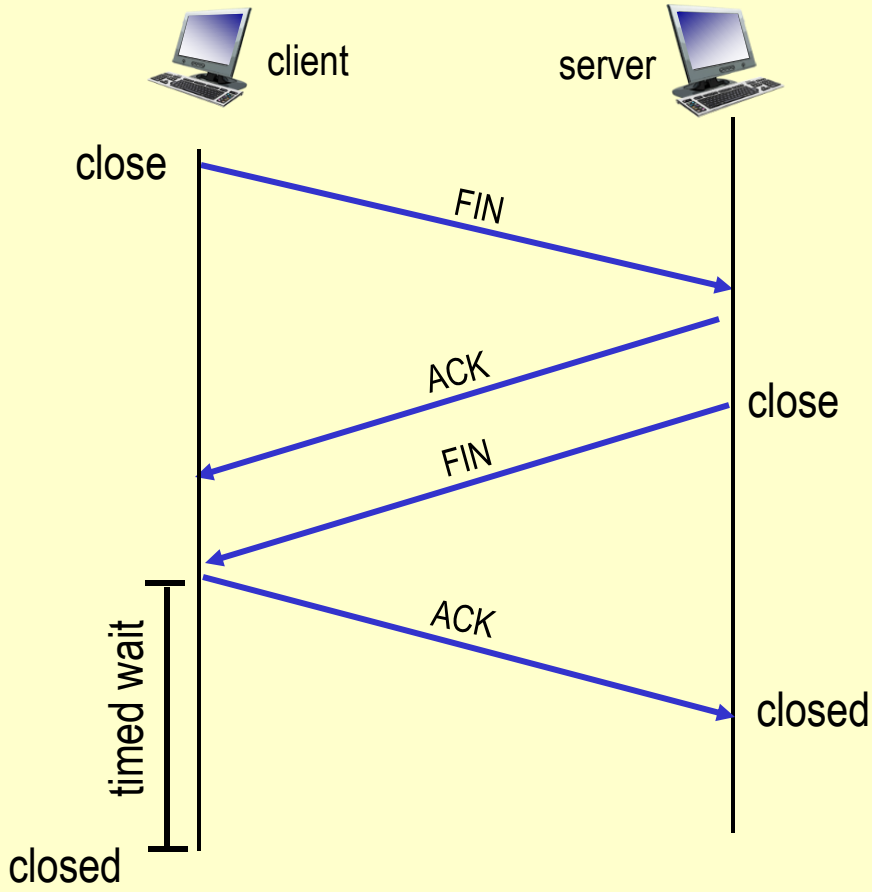


TCP Connection Management (cont.)

Step 3: client receives FIN, replies with ACK.

- enters “timed wait” - will respond with ACK to received FINs
- typically 30 seconds

Step 4: server, receives ACK. Connection closed.



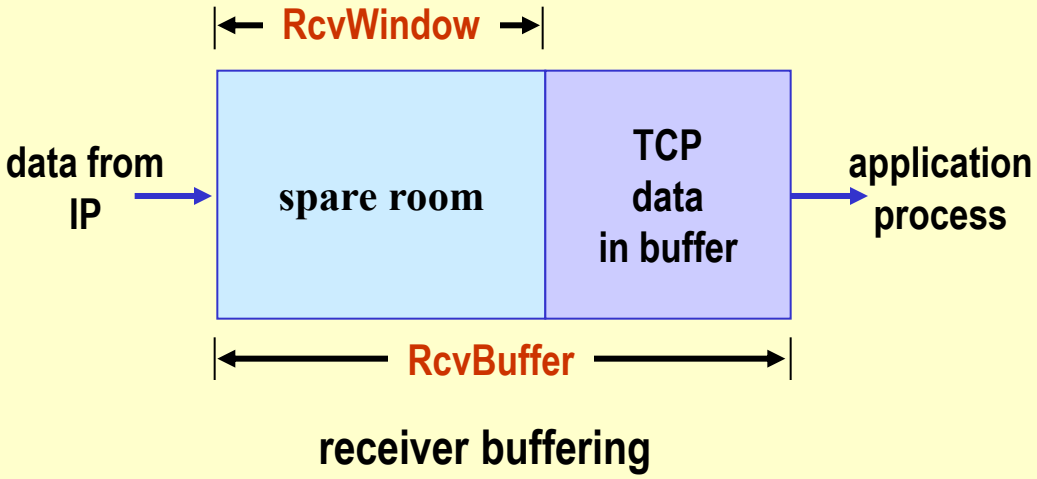
Today

- Transport layer
  - ✔ TCP – Connection Management
  - TCP – Flow Control
  - TCP – Congestion Control
  - TCP - Summary
- Network Layer
  - Recap – Review and Introduction
  - Routing – Link State (Dijkstra)
  - Routing – Distance Vector (Bellman Ford)
    - Basic algorithm
    - Cost change
    - Poisoned reverse
    - Real life solution
  - Comparing & Summary

TCP Flow Control

Sender won't overrun receiver's buffers by transmitting too much, too fast

`RcvBuffer` = size or TCP Receive Buffer
`RcvWindow` = amount of spare room in Buffer



receiver buffering

Receiver: explicitly informs sender of (dynamically changing) amount of free buffer space

- **`RcvWindow` field** in TCP segment

Sender: keeps the amount of transmitted, unACKed data less than most recently received **`RcvWindow`**

Today

- Transport layer
  - ✔ TCP – Connection Management
  - ✔ TCP – Flow Control
  - TCP – Congestion Control
  - TCP - Summary
- Network Layer
  - Recap – Review and Introduction
  - Routing – Link State (Dijkstra)
  - Routing – Distance Vector (Bellman Ford)
    - Basic algorithm
    - Cost change
    - Poisoned reverse
    - Real life solution
  - Comparing & Summary

TCP Congestion Control

Congestion:

- informally: "too many sources sending too much data too fast for *network* to handle"
- different from flow control!
- manifestations:
  - lost packets (buffer overflow at routers)
  - long delays (queuing in router buffers)

How does the sender perceive congestion?

- loss event = timeout *or* 3 duplicate acks

Reduce Window size:

- TCP sender reduces rate (**`CongWin`**) after loss event

Increase Widow size:

Two Phases

- slow start
- congestion avoidance – AIMD

Important variables:

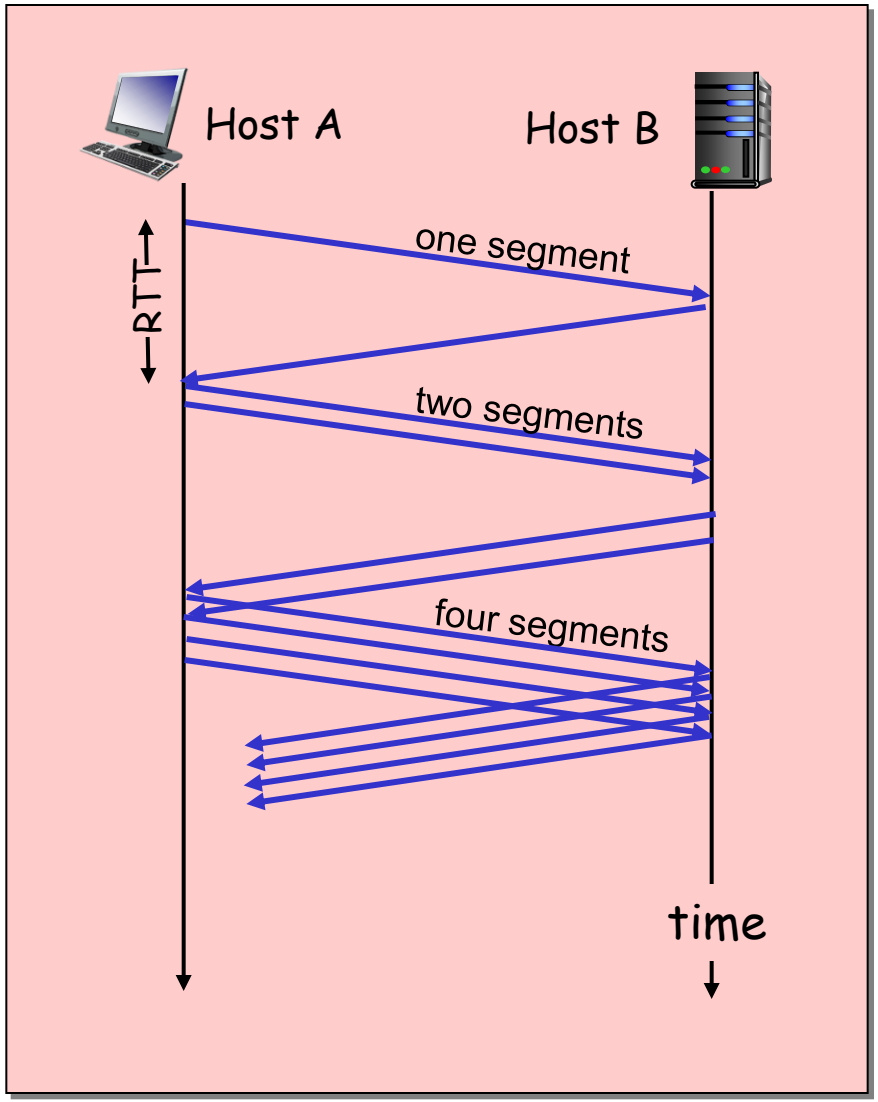
- **`Congwin`**
- **`threshold`**

TCP Slowstart

Slowstart algorithm

initialize: Congwin = 1
for (each segment ACKed)
    Congwin++
until (loss event OR
    CongWin > threshold)

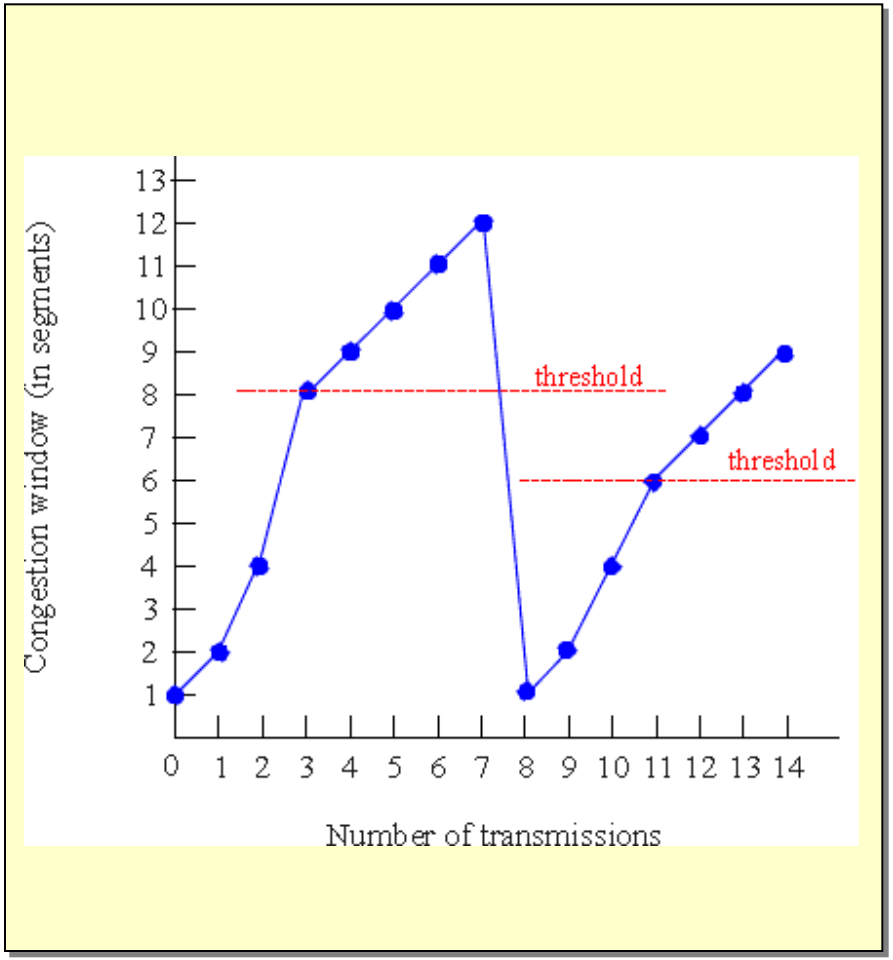
- initial rate is slow but ramps up exponentially fast
- loss event: timeout (Tahoe TCP) and/or three duplicate ACKs (Reno TCP)



TCP Congestion Avoidance

Congestion avoidance

```
/* slowstart is over        */
/* Congwin > threshold */
Until (loss event) {
   every w segments ACKed:
       Congwin++
   }
threshold = Congwin/2
Congwin = 1
perform slowstart[1]
```



TCP Reno skips slowstart (fast recovery) after three duplicate ACKs

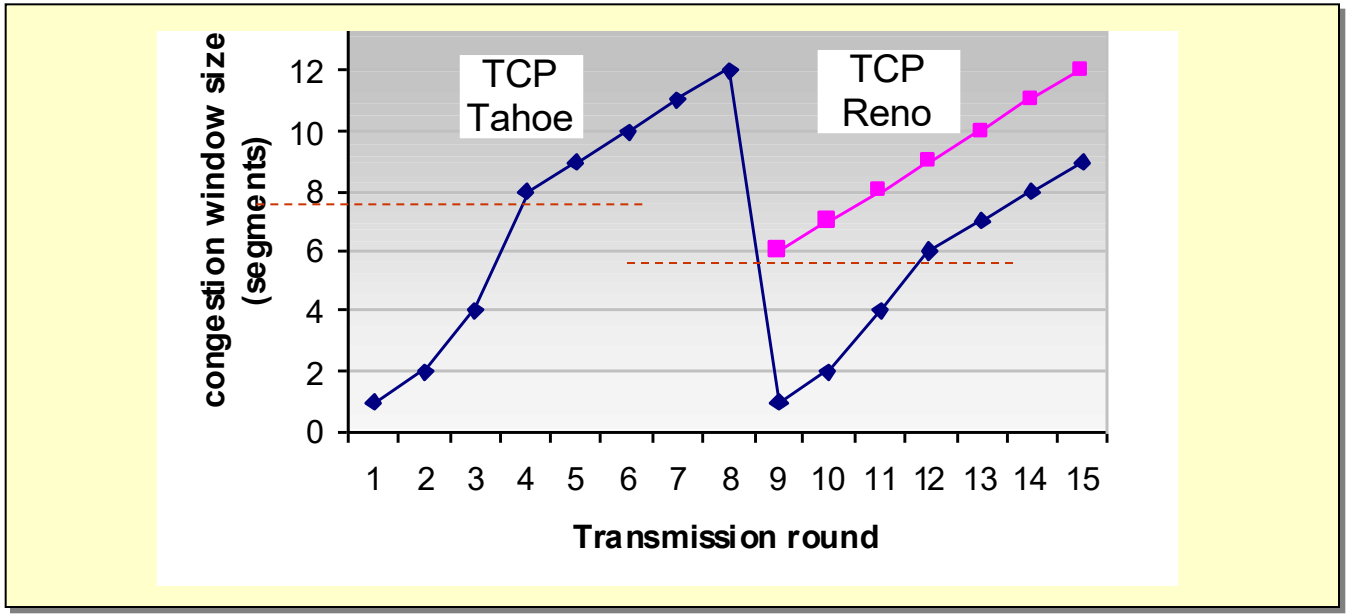
Refinement

- After 3 dup ACKs:
  - **`threshold`** is set to CongWin/2
  - **`CongWin`** is cut **in half**
  - window then grows linearly
- But after timeout event:
  - **`threshold`** is set to CongWin/2
  - **`CongWin`** is set to **1** MSS;
  - window then grows exponentially
  - to a threshold, then grows linearly

Philosophy

- 3 dup ACKs indicates network capable of delivering some segments
- timeout before 3 dup ACKs is "more alarming"

3 dup ACKS Tahoe vs. Reno



- TCP Tahoe – early version
  - unconditionally cuts congestion window to 1.
- TCP Reno – newer version
  - cancels the slow-start phase after a triple duplicate ACK.

The threshold initial value

Most modern TCP implementations often set the threshold to *infinite* (meaning a very high value like 2^31). Using an infinite value means that the congestion window will grow exponentially until a loss occurs. This way the optimal value is found more quickly.

Summary: TCP Congestion Control

- When **`CongWin`** is below **`Threshold`**, sender in slow-start phase, window grows exponentially.
- When **`CongWin`** is above **`Threshold`**, sender is in congestion-avoidance phase, window grows linearly.
- When a triple duplicate ACK occurs, **`Threshold`** set to **`CongWin/2`** and **`CongWin`** set to **`Threshold`**.
- When timeout occurs, **`Threshold`** set to **`CongWin/2`** and **`CongWin`** is set to 1 MSS.

Today

- Transport layer
  - ✔ TCP – Connection Management
  - ✔ TCP – Flow Control
  - ✔ TCP – Congestion Control
  - TCP - Summary
- Network Layer
  - Recap – Review and Introduction
  - Routing – Link State (Dijkstra)
  - Routing – Distance Vector (Bellman Ford)
    - Basic algorithm
    - Cost change
    - Poisoned reverse
    - Real life solution
  - Comparing & Summary

Problems with TCP

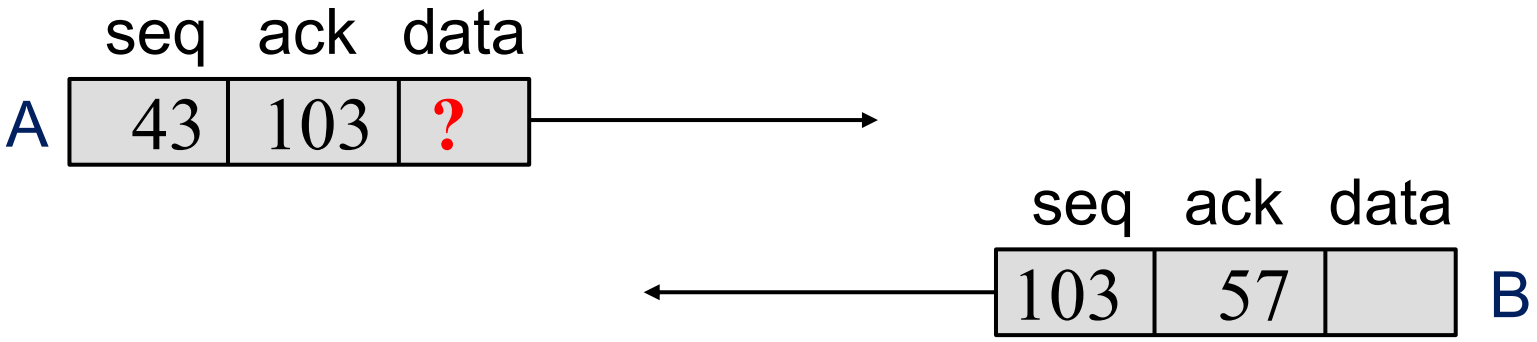
- Slow start
  - Small files doesn't get to full speed
- Head of line blocking
  - Won't deliver content after a lost packet
- Wireless loss does not imply congestion
  - Good for wired, less for wireless
- Slow handshake
  - Take a lot of time to establish a connection

https://www.youtube.com/watch?v=ejcESQOUQV8
(6:03 min)

שאלה

מחשב A שולח סיגמנט TCP למחשב B, בו המספר הסידורי הוא 43 וה- ACK הוא 103, ומחשב B שולח בתגובה סיגמנט בו המספר הסידורי הוא 103 וה- ACK הוא 57. כמה בתים של נתונים נשלחו בסיגמנט הראשון ?

Host A sends a TCP segment to host B, in which the sequence number is 43 and the ACK number is 103. Host B sends a segment in response with sequence number 103 and ACK number 57. How many bytes of data were sent in the first segment?



Question from an exam
Circle the correct answer

שאלה מבחינה (3 נקודות)
הקף בעיגול את התשובה המתאימה

במנגנון ה- congestion control ב- TCP Reno, כאשר ה- timer מצלצל אצל השולח, גודל threshold נקבע למחצית מערכו.

1. נכון
2. לא נכון

In TCP Reno congestion control mechanism, when the timer invokes an event at the sender's side, the threshold value is set to half its current value.

1. True
2. False

Question from an exam
Circle the correct answer

שאלה מבחינה (5 נקודות)
הקף בעיגול את התשובה המתאימה

השדה ב- TCP הנקרא RcvWindow (ז״א חלון הקבלה) משמש ל:

1. בקרת זרימה בלבד.
2. בקרת עומס בלבד.
3. בקרת זרימה וגם בקרת עומס.
4. גילוי שגיאות.
5. כל התשובות (1 עד 4) לא נכונות.

The RcvWindow field in TCP uses for:

1. Flow control only
2. Congestion control only
3. Flow control and congestion control
4. Error detection
5. Answers 1-4 are all false

Question from an exam
Circle the correct answer

שאלה מבחינה (5 נקודות)
הקף בעיגול את התשובה המתאימה

בשלב ה- slow start ב- TCP מתקיים:

1. גידול החלון איטי (לכל היותר ב- 1 במהלך RTT).
2. גידול החלון מהיר (החלון יכול להכפיל את גודלו במהלך RTT).
3. שלב זה אופציונאלי ותלוי בבררת המחדל של המשתמש.
4. מוקם קשר TCP עם חלון בגודל קבוע.
5. כל התשובות (1 עד 4) לא נכונות .

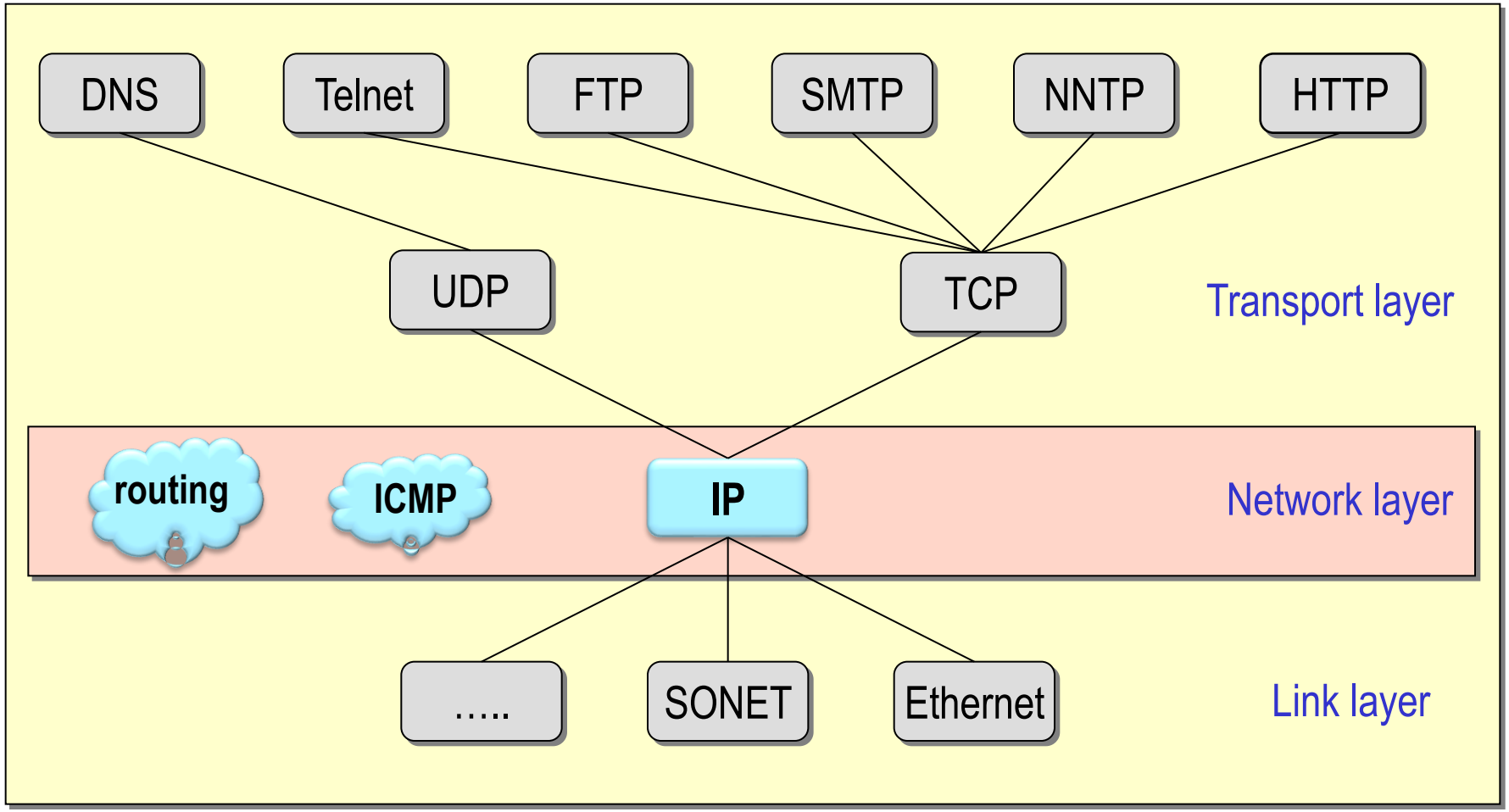
During slow start in TCP:

1. Window grows slowly (at most 1 each RTT)
2. Window grows fast (may double its size each RTT)
3. This phase is optional and determined by the user's set defaults
4. A TCP connection is established with a constant size window
5. Answers 1-4 are all false

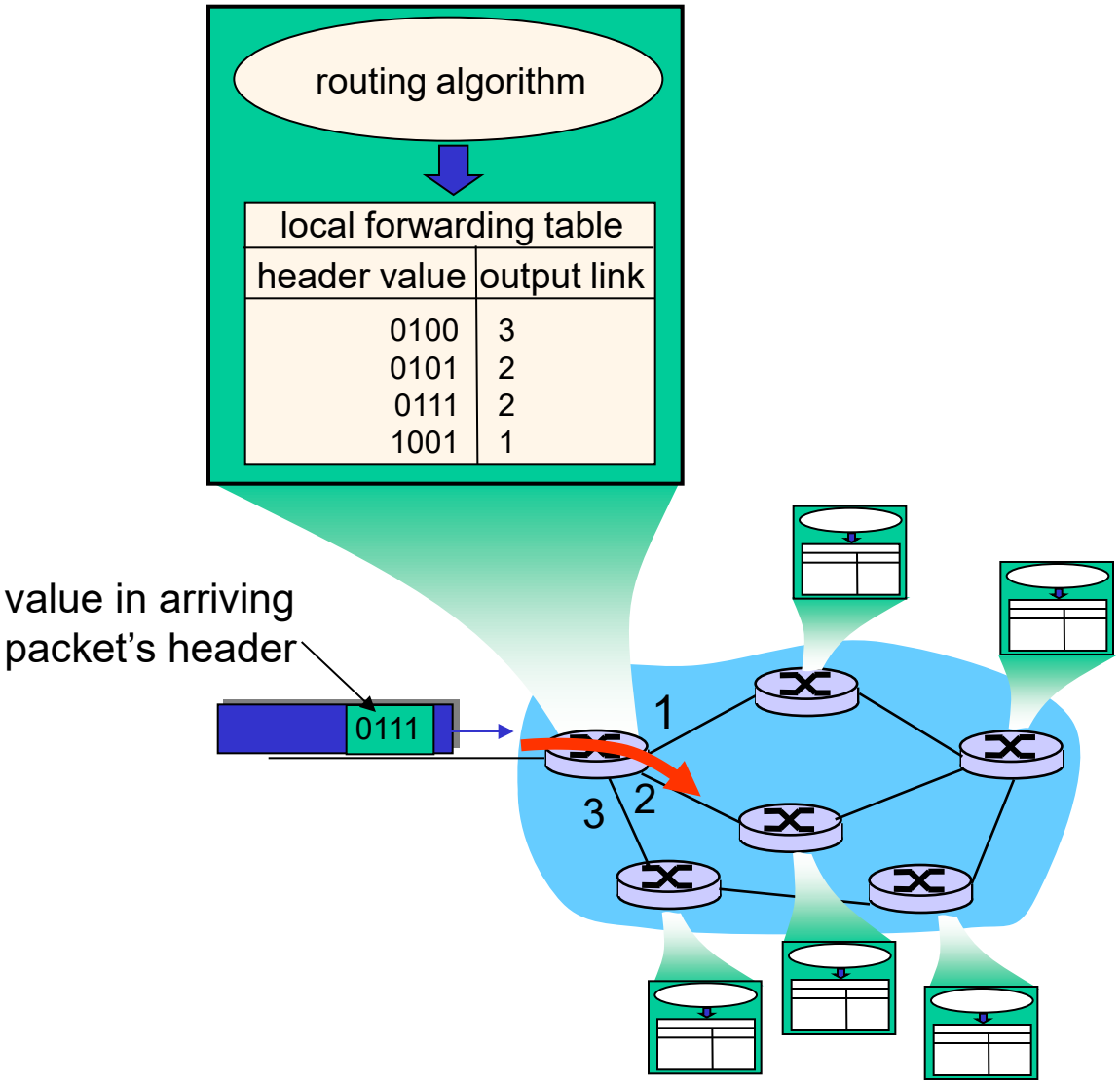
Today

- ✓ Transport layer
  - ✓ TCP – Connection Management
  - ✓ TCP – Flow Control
  - ✓ TCP – Congestion Control
  - ✓ TCP - Summary
- Network Layer
  - Recap – Review and Introduction
  - Routing – Link State (Dijkstra)
  - Routing – Distance Vector (Bellman Ford)
    - Basic algorithm
    - Cost change
    - Poisoned reverse
    - Real life solution
  - Comparing & Summary

Computer Networks



Interplay between routing and forwarding



Network Layer: Overview

The next few lectures

- Network layer services
- Routing principle --path selection
  - State-link (Dijkstra's Algorithm)
  - Distance-vector (Belman-ford)
  - Hierarchical routing
- Internet routing protocols
  - intra-domain (RIP, OSPF,EIGRP)
  - inter-domain (BGP)
- IP (Internet protocol)
- ICMP (Internet control message protocol)
- DHCP (Dynamic Host Configuration Protocol)
- NAT (Network Address Translators)
- Advanced topics: IPv6, Mobile IP, and multicast routing
- What's inside a router?

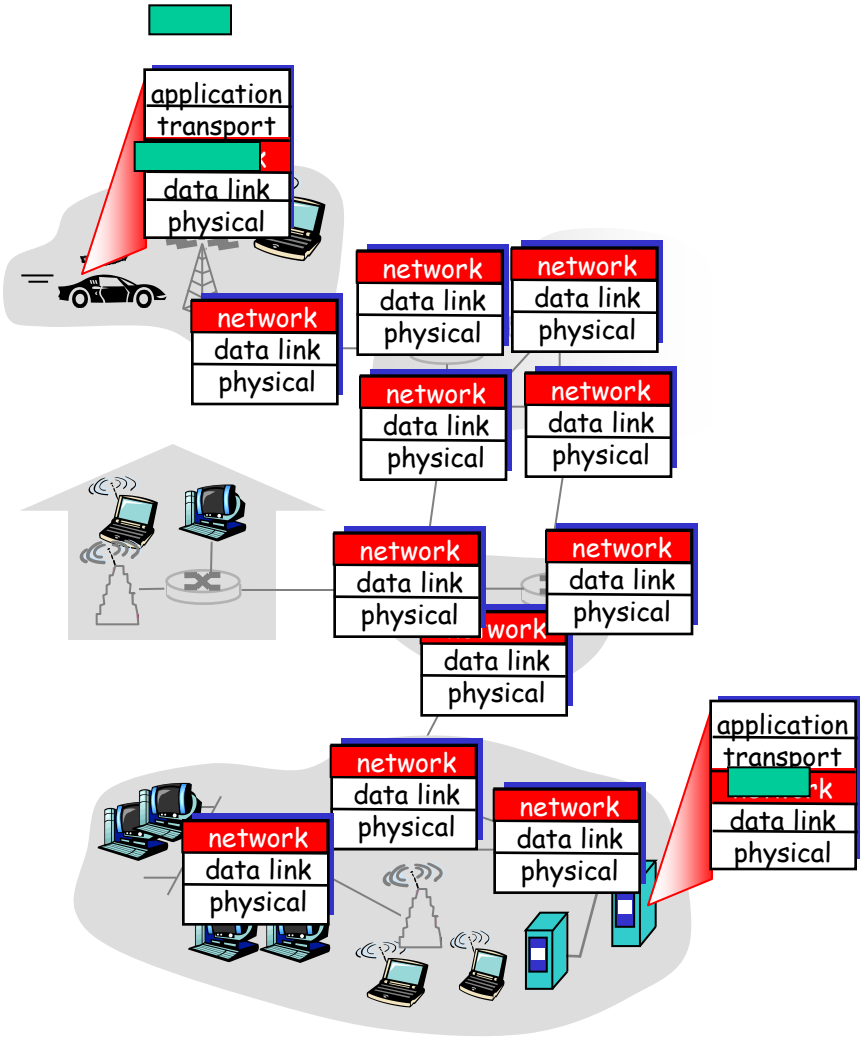
Network layer functions

Services

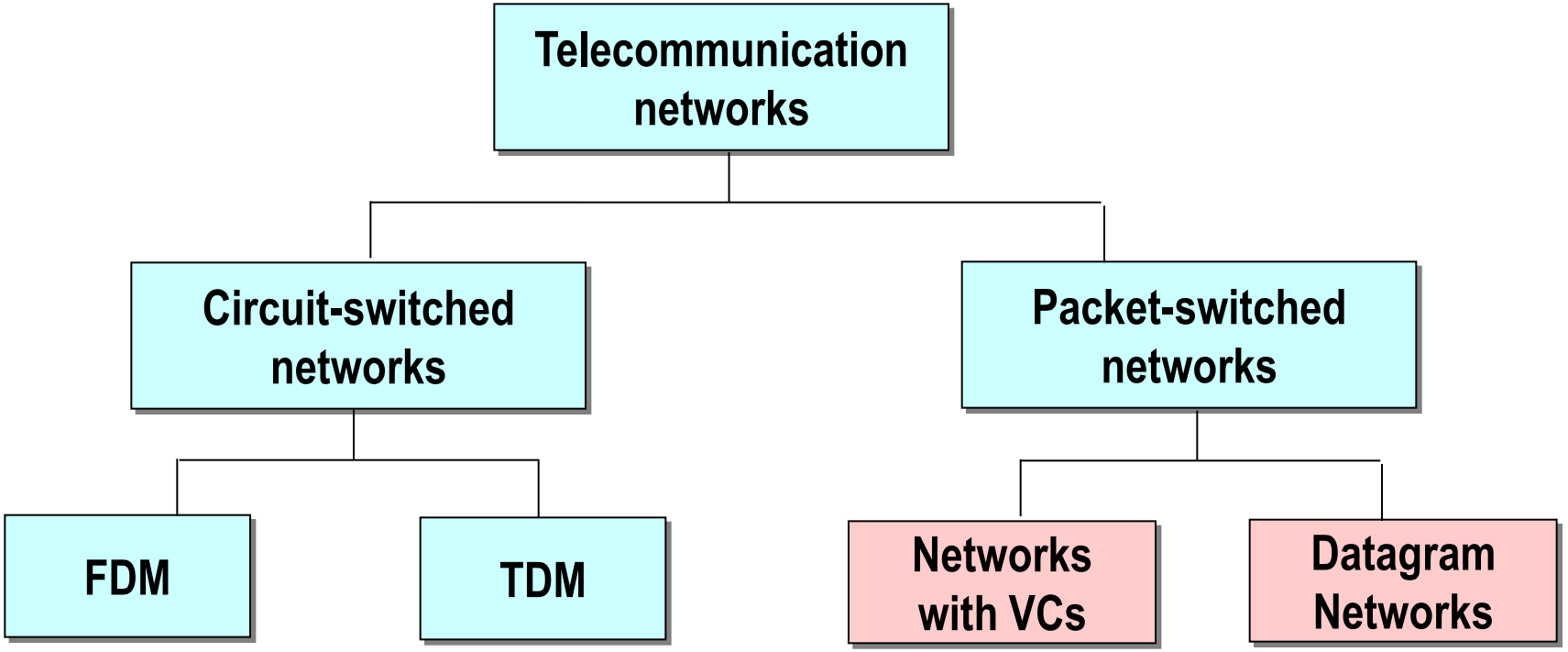
- transport packet from sending to receiving hosts
- network layer protocols in *every* host, and every router

Three important functions

- *routing/path determination:* route taken by packets from source to dest. *Routing algorithms*
- *forwarding/switching:* move packets from router's input to appropriate router output
- *call setup:* some network architectures require router call setup along path before data flows (not in IP)



Network Taxonomy



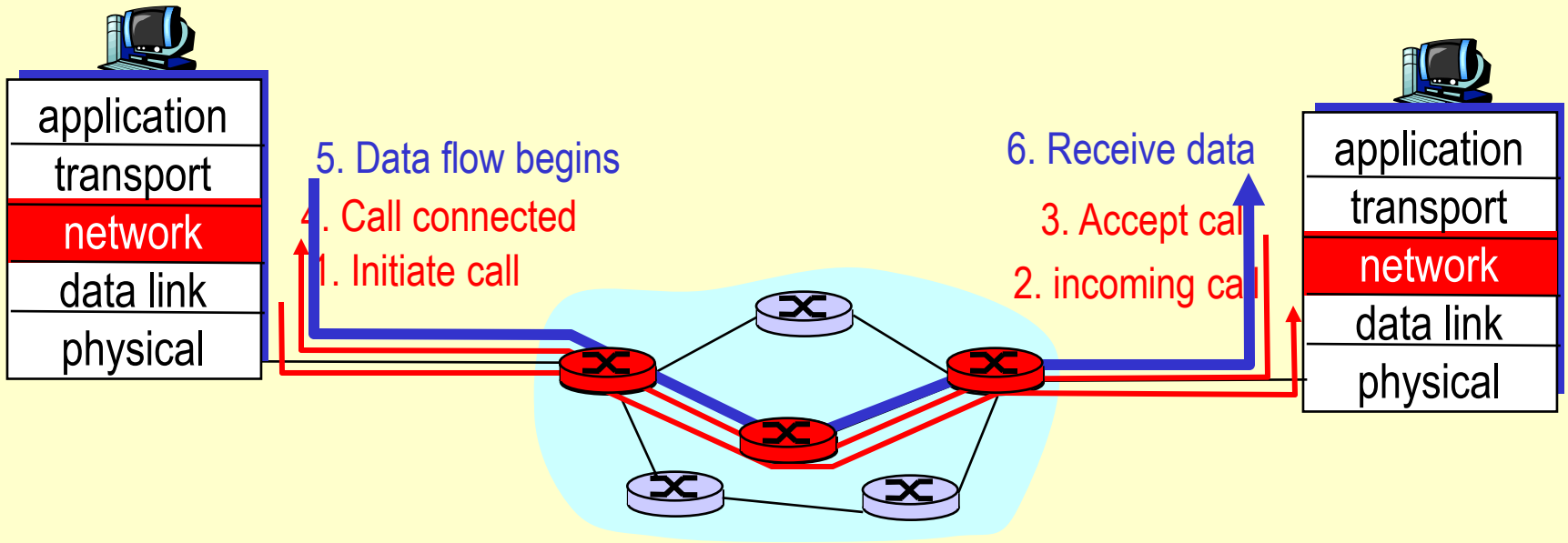
Virtual Circuits

Virtual circuits signaling protocols

- used to setup, maintain teardown VC
- used in ATM, frame-relay, X.25
- Almost not used in today's Internet

ATM

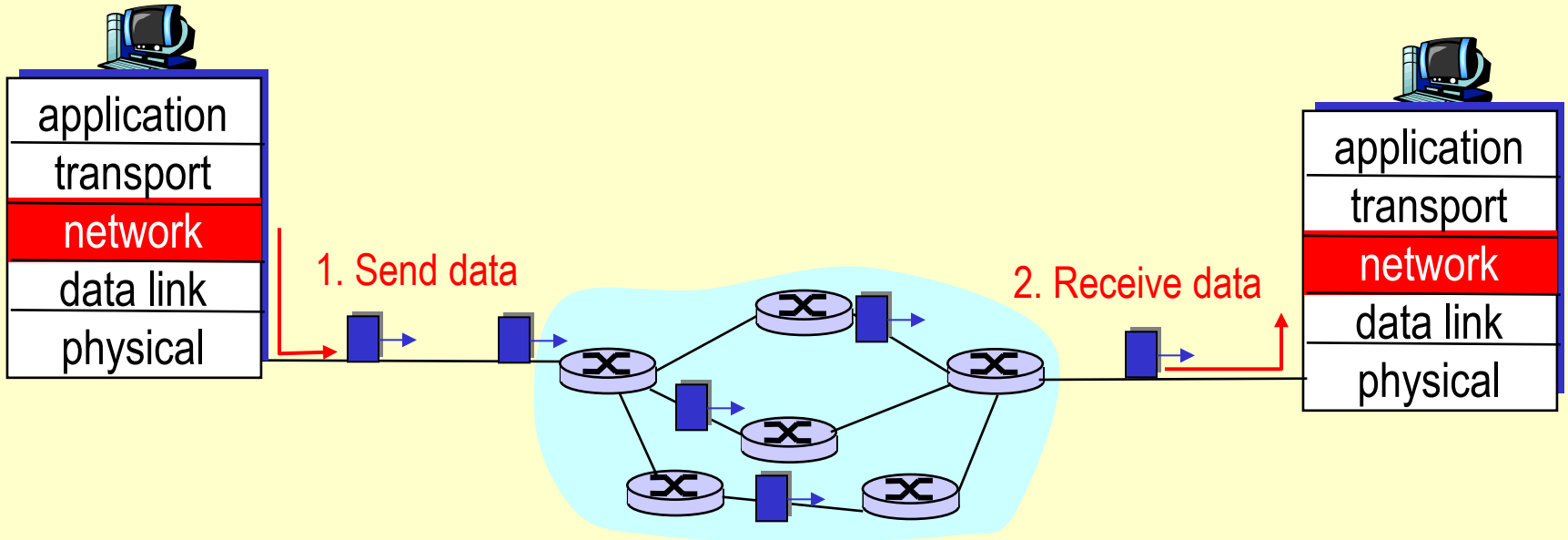
- "dumb" end systems (telephones); complexity inside network
- human conversation: strict timing requirements



Datagram networks: the Internet model

- no call setup at network layer
- no network-level concept of "connection"
- packets between same source-dest pair may take different paths

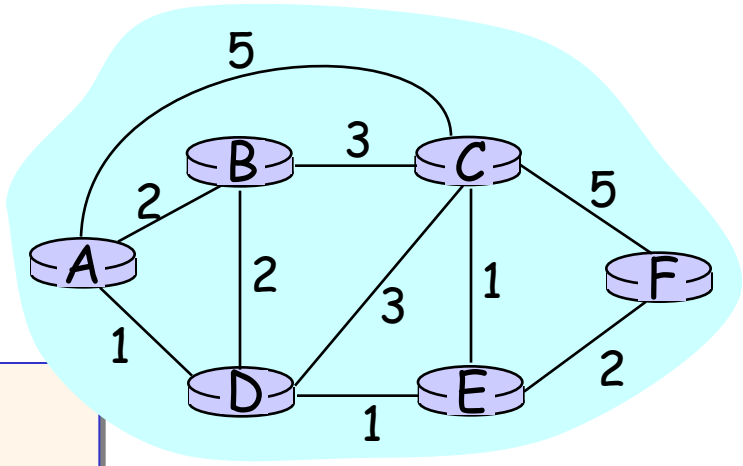
- "smart" end systems (computers); simple inside network
- data exchange among computers: no strict timing requirements



Routing

Routing protocol

Goal: determine "good" path (sequence of routers) thru network from source to dest.



Graph abstraction for routing algorithms:

- graph nodes are routers
- graph edges are physical links
  - link cost: delay, $ cost, or congestion level
- "good" path:
  - typically means minimum cost path
  - other def's possible

Routing algorithm classification

Global or decentralized information?

Global

- all routers have complete topology, link cost info
- "link state" algorithms – Dijkstra algorithm

Decentralized

- router knows physically-connected neighbors, link costs to neighbors
- "distance vector" algorithms – Bellman-Ford (not Floyd-Warshall)

Today

- ✓ Transport layer
  - ✓ TCP – Connection Management
  - ✓ TCP – Flow Control
  - ✓ TCP – Congestion Control
  - ✓ TCP - Summary
- Network Layer
  - ✓ Recap – Review and Introduction
  - Routing – Link State (Dijkstra)
  - Routing – Distance Vector (Bellman Ford)
    - Basic algorithm
    - Cost change
    - Poisoned reverse
    - Real life solution
  - Comparing & Summary

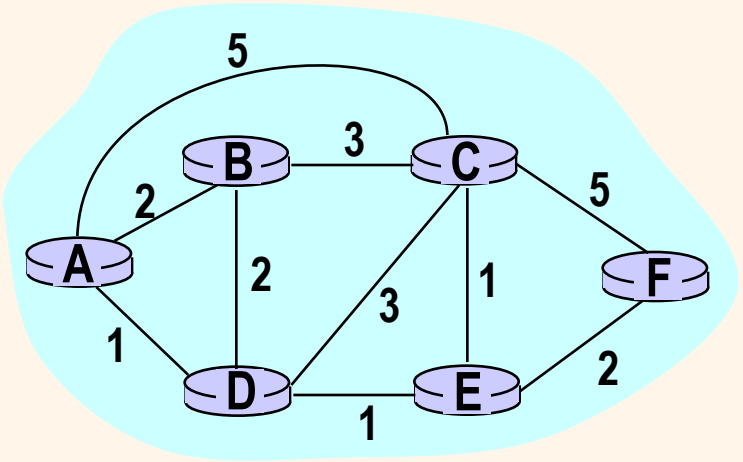
A Link-State Routing Algorithm

Dijkstra's algorithm

- net topology, link costs known to all nodes
  - accomplished via "link state broadcast"
  - all nodes have same info
- computes least cost paths from one node ('source") to all other nodes
  - gives routing table for that node

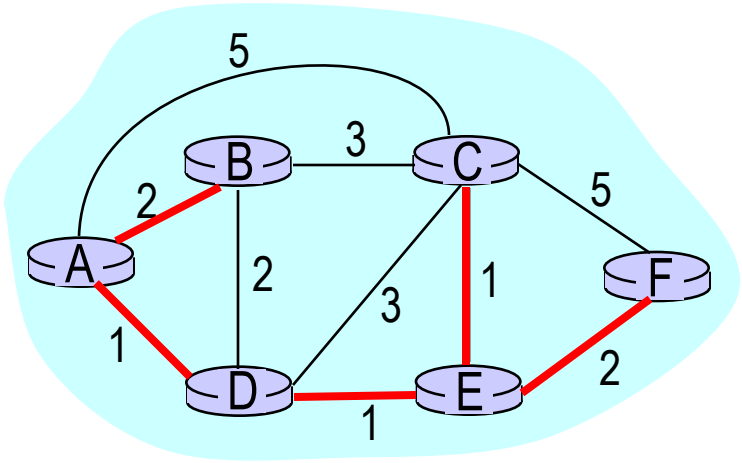
Notation

- c(i,j): link cost from node **i** to **j**. cost infinite if not direct neighbors
- D(v): current value of cost of path from source to dest. **V**
- p(v): predecessor node along path from source to v, that is next **V**



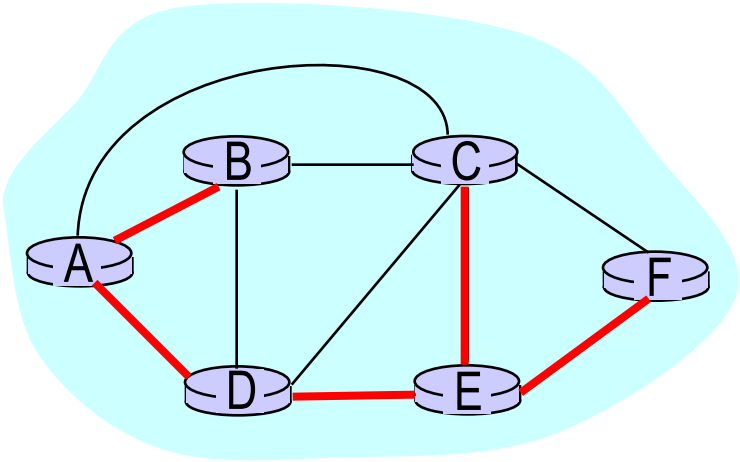
Dijkstra's algorithm: example

Step	start N	D(B),p(B)	D(C),p(C)	D(D),p(D)	D(E),p(E)	D(F),p(F)
0	A	2,A	5,A	1,A	infinity	infinity
1	AD	2,A	4,D		2,D	infinity
2	ADE	2,A	3,E			4,E
3	ADEB		3,E			4,E
4	ADEBC					4,E
5	ADEBCF					



Dijkstra's algorithm: example

Resulting shortest-path tree from A :



Resulting forwarding table in A :

destination	link
B	(A,B)
D	(A,D)
E	(A,D)
C	(A,D)
F	(A,D)

Dijsktra's Algorithm

```
1  Initialization:
2   N = {A}     /* set of nodes whose least cost path definitively known */
3   for all nodes v
4     if v adjacent to A
5       then D(v) = c(A,v)
6       else D(v) = infty
7
8  Loop
9    find w not in N such that D(w) is a minimum
10   add w to N
11   update D(v) for all v adjacent to w and not in N:
12      D(v) = min( D(v), D(w) + c(w,v) )
13      /* new cost to v is either old cost to v or known
14      shortest path cost to w plus cost from w to v */
15 until all nodes in N
```

Question from an exam
Circle the correct answer

שאלה מבחינה (3 נקודות)
הקף בעיגול את התשובה המתאימה

העץ הפורש הנבנה על ידי אלגוריתם דייקסטרה הוא עץ פורש מינימלי.

1. נכון
2. לא נכון

The spanning tree resulting by the Dijkstra algorithm is a minimal spanning tree.

1. True
2. False

Question from an exam
Circle the correct answer

שאלה מבחינה (3 נקודות)
הקף בעיגול את התשובה המתאימה

שני צמתים שכנים באותה רשת תקשורת המריצים את אלגוריתם ה- link-state (דיקסטרה) יבנו את אותו העץ הפורש.

1. נכון
2. לא נכון

Two adjacent neighbors on the same network which run the link-state algorithm will independently construct the same spanning tree.

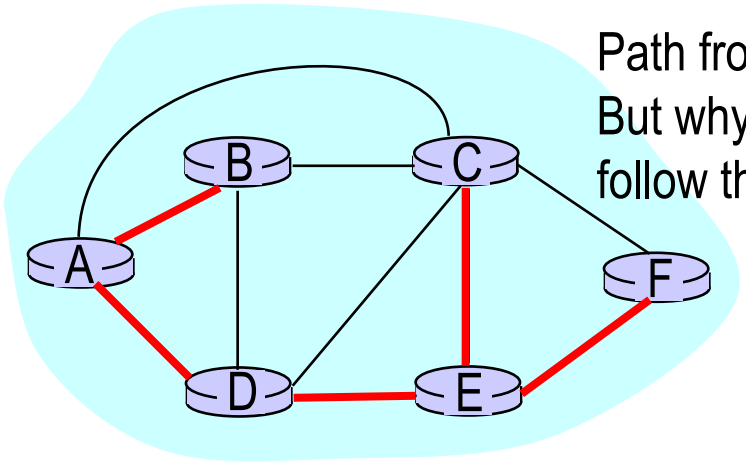
1. True
2. False

Link-State: Why should it work?

Re-think the question in the previous slide:

Two adjacent nodes construct different spanning trees.

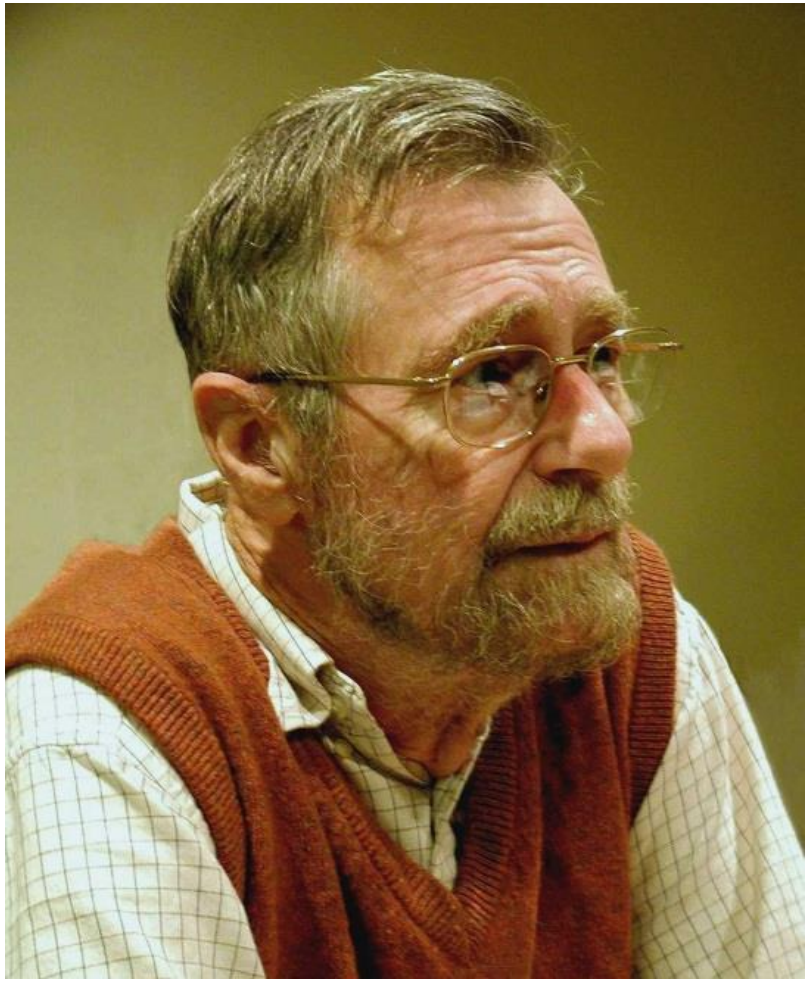
Then how can node A trust the next hop to forward a packet towards the same path chosen by A?



Path from A to F is ADEF. But why will next-hop D follow this path?

Hint: Use exchange argument for the part of the path ‘D to F’.

Edsger W. Dijkstra



1930 -- 2002

www.cs.utexas.edu/users/EWD/

(Photo by *Hamilton Richards* 2002)

Today

- ✓ Transport layer
  - ✓ TCP – Connection Management
  - ✓ TCP – Flow Control
  - ✓ TCP – Congestion Control
  - ✓ TCP - Summary
- Network Layer
  - ✓ Recap – Review and Introduction
  - ✓ Routing – Link State (Dijkstra)
  - Routing – Distance Vector (Bellman Ford)
    - Basic algorithm
    - Cost change
    - Poisoned reverse
    - Real life solution
  - Comparing & Summary

Distance Vector Routing Algorithm

Iterative

- continues until no nodes exchange info.
- *self-terminating*: no “signal” to stop

Asynchronous

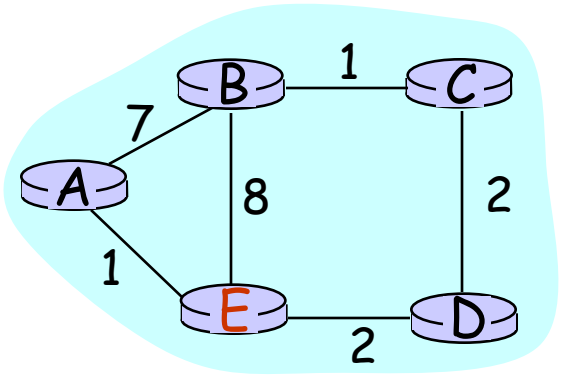
- nodes need *not* exchange info/iterate in lock step!

Distributed

- each node communicates *only* with directly-attached neighbors

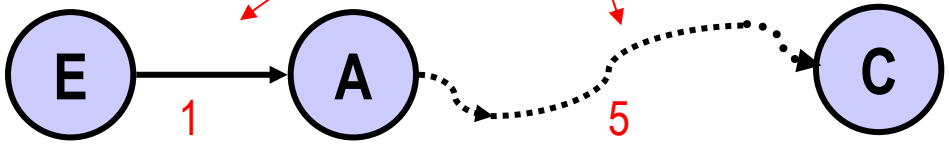
Distance Vector Routing Algorithm

Distance table data structure



$D^E(C,A)$ = distance *from* E *to* C, *via* A as next hop

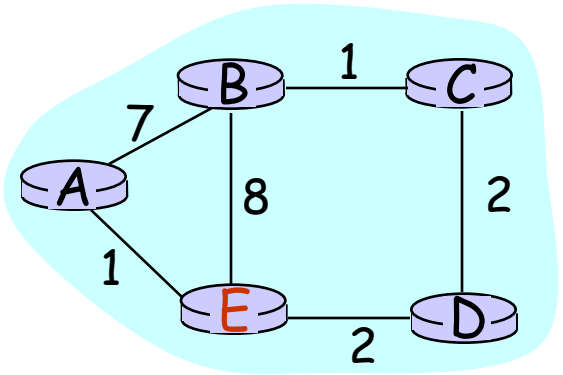
$= c(E,A) + \min_w\{D^A(C,w)\}$



$D^E()$ — cost to destination via

destination	A	B	D
A	1	14	5
B	7	8	5
C	6	9	4
D	4	11	2

Distance Table: example



$$D^E(C,D) = c(E,D) + \min_w\{D^D(C,w)\}$$
$$= 2+2 = 4$$

$$D^E(A,D) = c(E,D) + \min_w\{D^D(A,w)\}$$
$$= 2+3 = 5$$ loop!

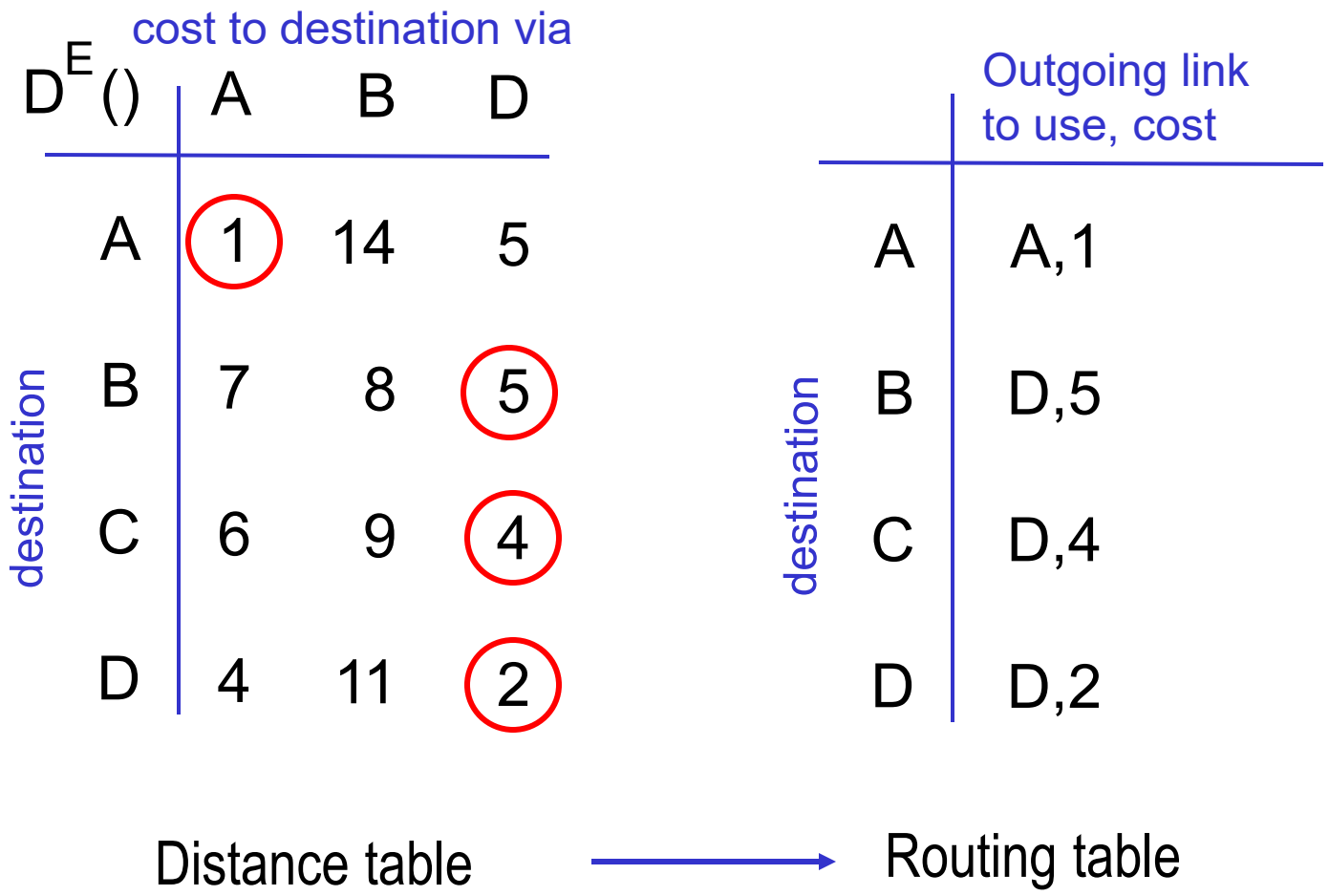
$$D^E(A,B) = c(E,B) + \min_w\{D^B(A,w)\}$$
$$= 8+6 = 14$$ loop!

cost to destination via

$D^E()$	A	B	D
A	1	14	5
B	7	8	5
C	6	9	4
D	4	11	2

destination

Distance table gives routing table



Distance Vector Routing: overview

Iterative, asynchronous:

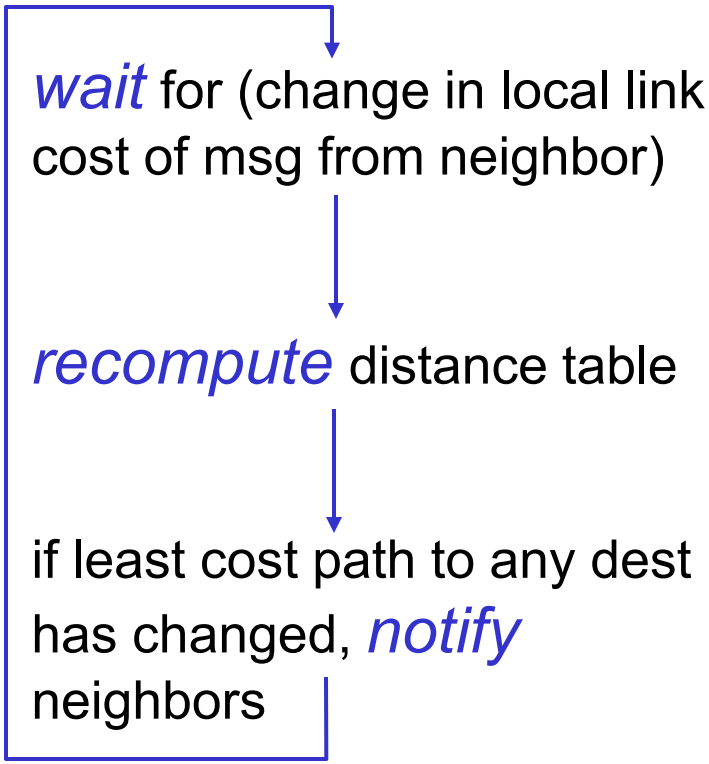
each local iteration caused by:

- local link cost change
- message from neighbor: its least cost path change from neighbor

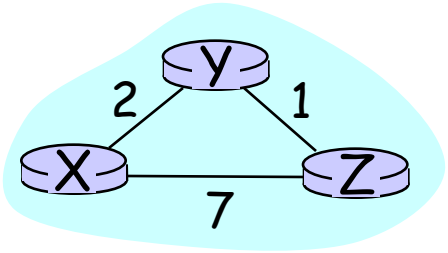
Distributed:

- each node notifies neighbors *only* when its least cost path to any destination changes
  - neighbors then notify their neighbors if necessary

Each node:



Distance Vector Algorithm: example



D^X

dest	via Y	via Z
Y	2	∞
Z	∞	7

D^Y

dest	via X	via Z
X	2	∞
Z	∞	1

D^Z

dest	via X	via Y
X	7	∞
Y	∞	1

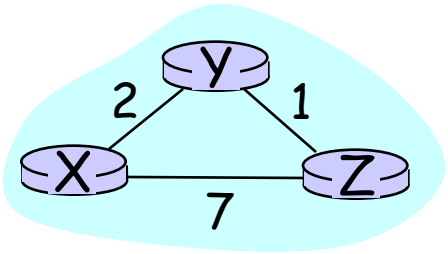
D^X

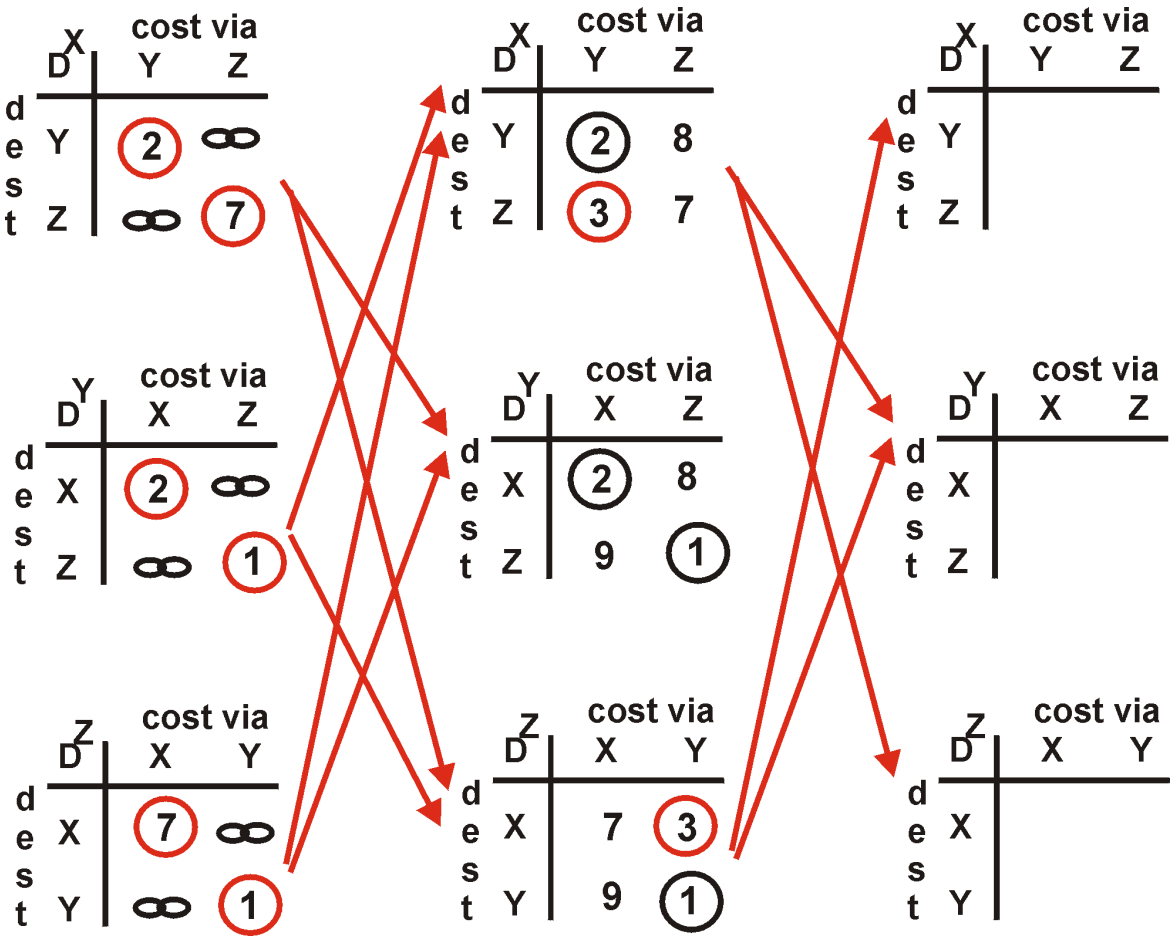
dest	via Y	via Z
Y	2	8
Z	3	7

$$D^X(Y,Z) = c(X,Z) + \min_w\{D^Z(Y,w)\}$$
$$= 7+1 = 8$$

$$D^X(Z,Y) = c(X,Y) + \min_w\{D^Y(Z,w)\}$$
$$= 2+1 = 3$$

Distance Vector Algorithm: example





Distance Vector Algorithm:

At all nodes, X:

1 Initialization:
2 for all adjacent nodes v:
3 $D^X(*,v) = \text{infty}$ /* the * operator means "for all rows" */
4 $D^X(v,v) = c(X,v)$
5 for all destinations, y
6 send $\min_w D^X(y,w)$ to each neighbor /* w over all X's neighbors */

Distance Vector Algorithm (cont.):

```
8  loop
9    wait (until I see a link cost change to neighbor V
10         or until I receive update from neighbor V)
11
12   if (c(X,V) changes by d)
13     /* change cost to all dest's via neighbor v by d */
14     /* note: d could be positive or negative */
15     for all destinations y:  D^X(y,V) =  D^X(y,V) + d
16
17   else if (update received from V wrt destination Y)
18     /* shortest path from V to some Y has changed  */
19     /* V has sent a new value for its  min_w DV(Y,w) */
20     /* call this received new value is "newval"     */
21     for the single destination y: D^X(Y,V) = c(X,V) + newval
22
23   if we have a new min_w D^X(Y,w)for any destination Y
24      send new value of min_w D^X(Y,w) to all neighbors
25 forever
```

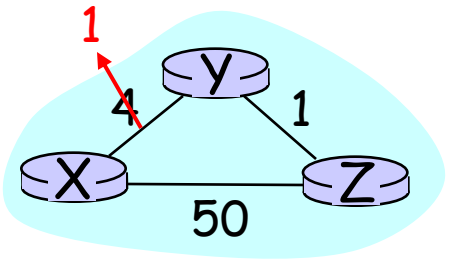
Today

- ✓ Transport layer
  - ✓ TCP – Connection Management
  - ✓ TCP – Flow Control
  - ✓ TCP – Congestion Control
  - ✓ TCP - Summary
- Network Layer
  - ✓ Recap – Review and Introduction
  - ✓ Routing – Link State (Dijkstra)
  - Routing – Distance Vector (Bellman Ford)
    - ✓ Basic algorithm
    - Cost change
    - Poisoned reverse
    - Real life solution
  - Comparing & Summary

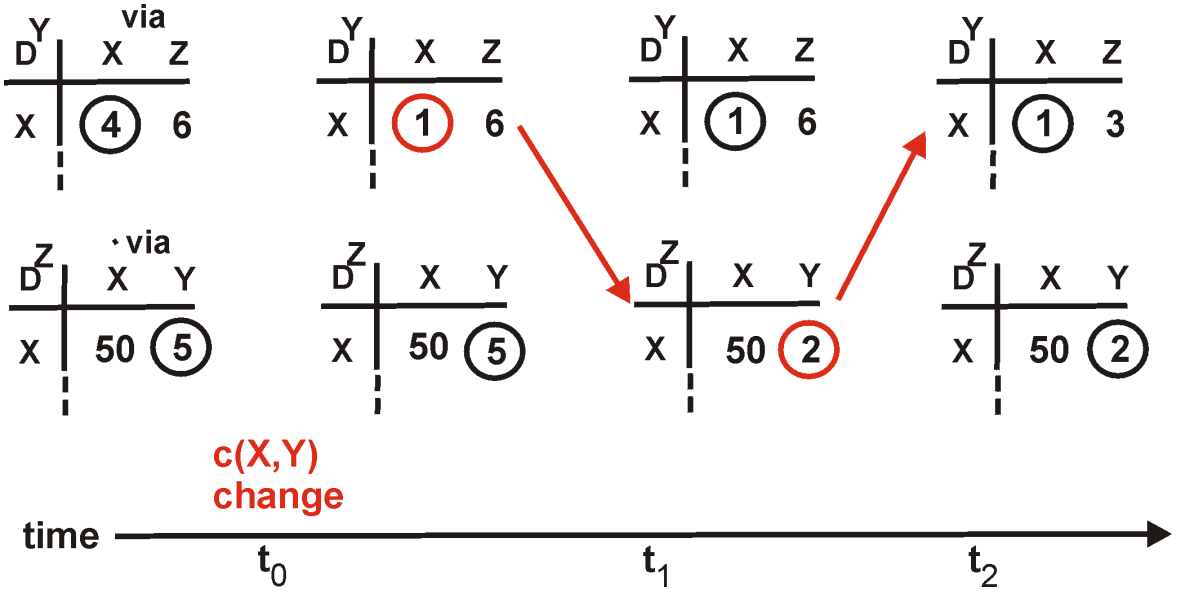
Distance Vector: link cost changes

Link cost changes:

- node detects local link cost change
- updates distance table (line 15)
- if cost change in least cost path, notify neighbors (lines 23,24)



"good news travels fast"

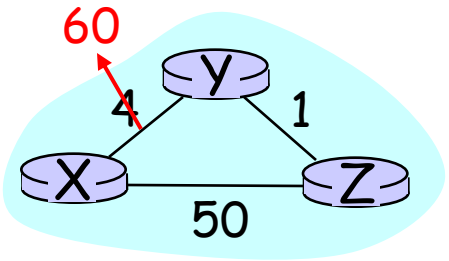


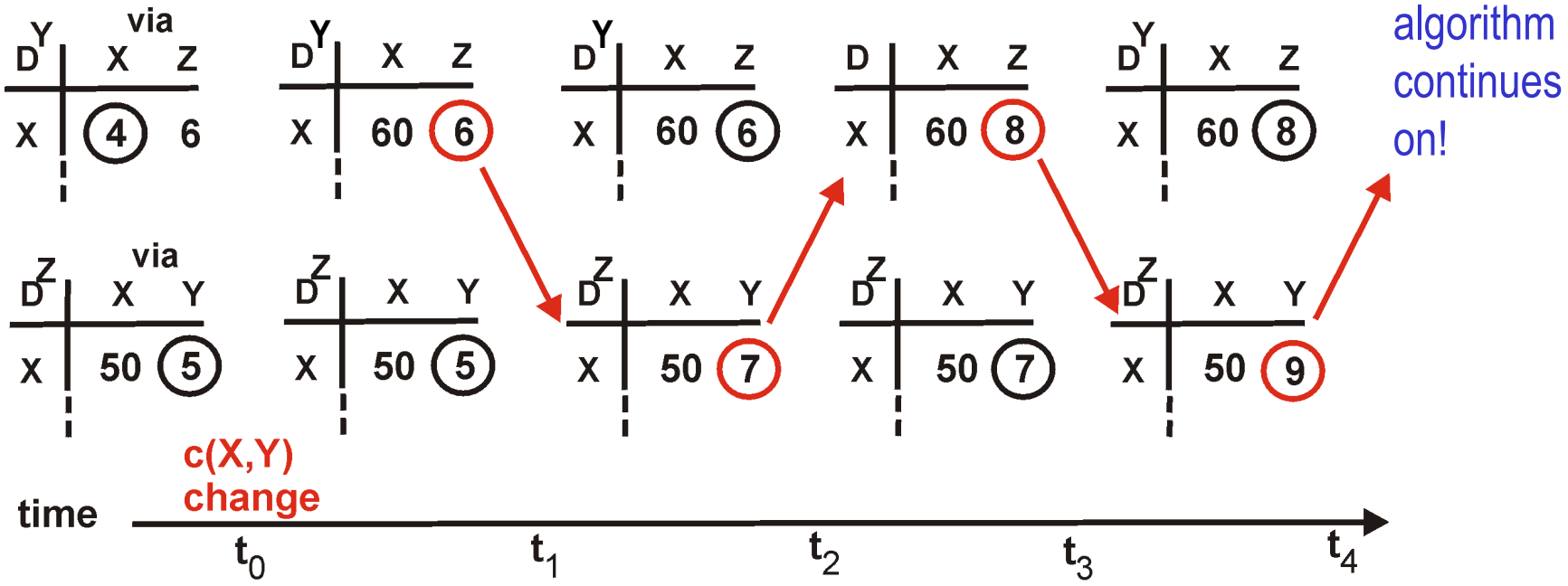
algorithm terminates

Distance Vector: link cost changes

Link cost changes:

- good news travels fast
- bad news travels slow – "count to infinity" problem!





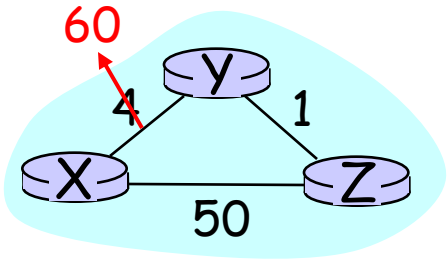
Today

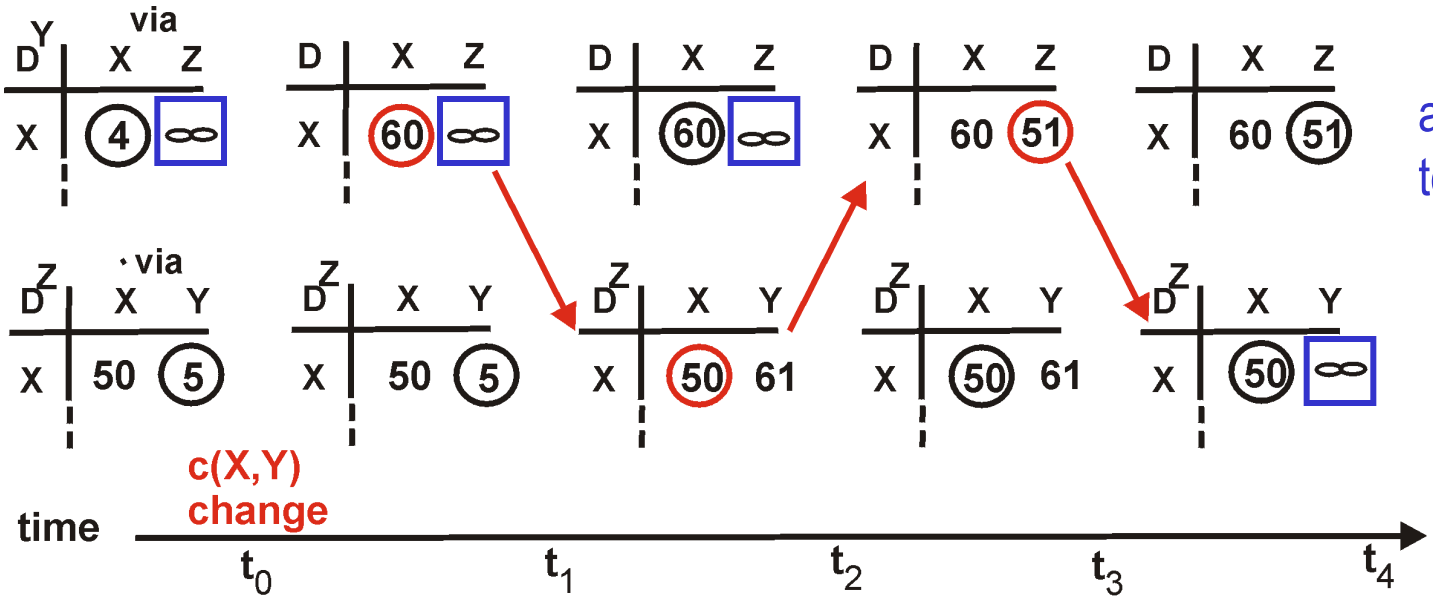
- ✓ Transport layer
  - ✓ TCP – Connection Management
  - ✓ TCP – Flow Control
  - ✓ TCP – Congestion Control
  - ✓ TCP - Summary
- Network Layer
  - ✓ Recap – Review and Introduction
  - ✓ Routing – Link State (Dijkstra)
  - Routing – Distance Vector (Bellman Ford)
    - ✓ Basic algorithm
    - ✓ Cost change
    - Poisoned reverse
    - Real life solution
  - Comparing & Summary

Distance Vector: poisoned reverse

If Z routes through Y to get to X :

- Z tells Y its (Z's) distance to X is infinite (so Y won't route to X via Z)
- will this completely solve count to infinity problem?





algorithm terminates

Distance Vector Algorithm:

At all nodes, X:

```
1  Initialization:
2   for all adjacent nodes v:
3      D^X(*,v) = infty        /* the * operator means "for all rows" */
4      D^X(v,v) = c(X,v)
5   for all destinations, y
6      send min_w D^X(y,w) to each neighbor  /* w over all X's neighbors */
```

Distance Vector Algorithm (cont.):

```
8 loop
9    wait (until I see a link cost change to neighbor V
10         or until I receive update from neighbor V)

12   if (c(X,V) changes by d)
13     /* change cost to all dest's via neighbor v by d */
14     /* note: d could be positive or negative */
15     for all destinations y:
16        D^X(y,V) = D^X(y,V) + d

17   else if (update received from V wrt destination Y)
18     /* shortest path from V to some Y has changed */
19     /* V has sent a new value for its min_W DV(Y,w) */
20     /* call this received new value is "newval" */
21     D^X(Y,V) = c(X,V) + newval

23   if we have a new min_W D^X(Y,w) in any destination Y
24     for each neighbor N:
25        for each destination Y:
26           if (nextHop(Y) == N)
27              send to N: D(Y) = ∞      /* poisoned reverse */
28           else
29              send to N: D(Y) = min_W D^X(Y,w)
30 forever
```

Today

- ✓ Transport layer
  - ✓ TCP – Connection Management
  - ✓ TCP – Flow Control
  - ✓ TCP – Congestion Control
  - ✓ TCP - Summary
- Network Layer
  - ✓ Recap – Review and Introduction
  - ✓ Routing – Link State (Dijkstra)
  - Routing – Distance Vector (Bellman Ford)
    - ✓ Basic algorithm
    - ✓ Cost change
    - ✓ Poisoned reverse
    - Real life solution
  - Comparing & Summary

Distance Vector: Real Life

Aim: reduce instability, not eliminate it entirely

- Poisoned reverse is not sufficient alone
- Protocols add multiple mechanisms:
  - Split horizon
  - Triggered updates
  - Hold-down timers
  - Bounded “infinity” values
- Cannot create a network that does both Dijkstra and Distance Vector

Today

- ✓ Transport layer
  - ✓ TCP – Connection Management
  - ✓ TCP – Flow Control
  - ✓ TCP – Congestion Control
  - ✓ TCP - Summary
- Network Layer
  - ✓ Recap – Review and Introduction
  - ✓ Routing – Link State (Dijkstra)
  - ✓ Routing – Distance Vector (Bellman Ford)
    - ✓ Basic algorithm
    - ✓ Cost change
    - ✓ Poisoned reverse
    - ✓ Real life solution
  - Comparing & Summary

Comparison of LS and DV algorithms

Message

- LS (Dijkstra): Initially O(nE) messages – each node has to know each link.
  Sent update messages to all nodes.
- DV: exchange between neighbors only (convergence time varies).

Robustness: **What happens if router malfunctions?**

- LS: node can advertise incorrect *link* cost,
  - each node computes only its *own* table
- DV: DV node can advertise incorrect *path* cost !!!
  - each node's table used by others (error propagate thru network)

Today

- ✓ Transport layer
  - ✓ TCP – Connection Management
  - ✓ TCP – Flow Control
  - ✓ TCP – Congestion Control
  - ✓ TCP - Summary
- Network Layer
  - ✓ Recap – Review and Introduction
  - ✓ Routing – Link State (Dijkstra)
  - ✓ Routing – Distance Vector (Bellman Ford)
    - ✓ Basic algorithm
    - ✓ Cost change
    - ✓ Poisoned reverse
    - ✓ Real life solution
  - ✓ Comparing & Summary

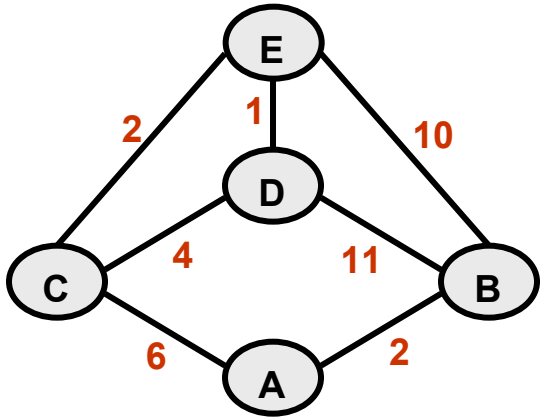
Distance Vector

A question from an exam

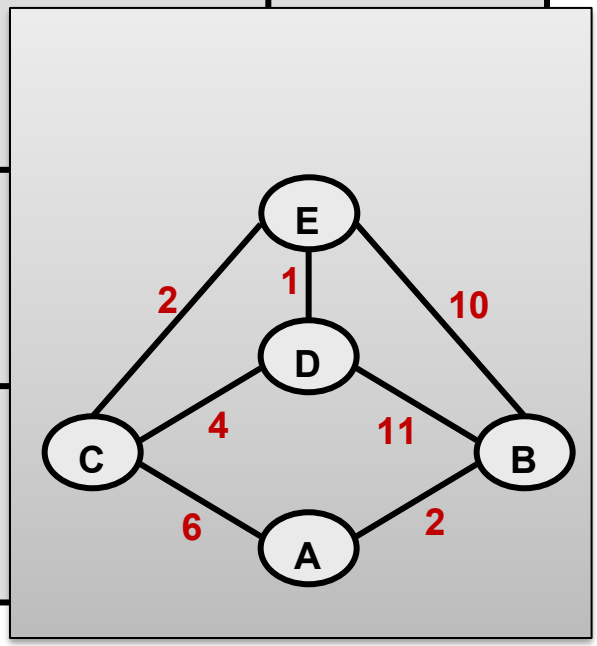
cost to destination via

destination

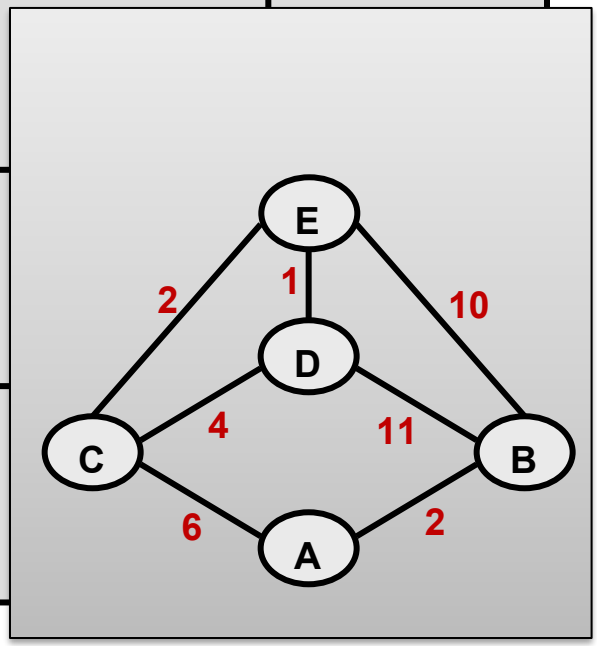
table of A	B	C
B	2	∞
C	∞	6
D	∞	∞
E	∞	∞



	t = 0	t = 1	t = 2	t = 3	t = 4	t = 5
	initial state	E→ D	D→ C	C→ D	D→ B	B→ A
A	B: 2; C: 6					B, C
B	A, D, E				A, D, E	
C	A, D, E		A, D, E			
D	B, C, E	B, C, E		B, C, E		
E	B, C, D					



	t = 0	t = 1	t = 2	t = 3	t = 4	t = 5
	initial state	E→ D	D→ C	C→ D	D→ B	B→ A
A	B: B 2, C 6					B: B 2, C 6, D 13, E 12
B	A: A 2, D 11, E 10				A: A 2, D 21, C 14, D 11, E 12	
C	A: A 6, D 4, E 2		A: A 6, D 15, D 4, E 5			
D	B: B 11, C 4, E 1	B: B 11, C 4, E 11, E 1		B: B 11, C 10, C 4, E 6		
E	B: B 10, C 2, D 1					



Question from an exam
Circle the correct answer

שאלה מבחינה (3 נקודות)
הקף בעיגול את התשובה המתאימה

בשאלה זו, ובבאות אחריה, הניחו שיכולות להיות בגרף קשתות מקבילות.

אם הניתוב הוא לפי וקטור-מרחק, אזי כאשר יש שינוי כלשהו במשקל של אחת מהקשתות, תמיד צומת אחד לפחות חייב לשלוח הודעת עדכון.

1. נכון
2. לא נכון

In this question and the next ones, assume there can be parallel edges.

In distance-vector routing, when there is a change in the weight of one of the edges, at least one node must always send an update message.
1. True
2. False

Question from an exam
Circle the correct answer

שאלה מבחינה (3 נקודות)
הקף בעיגול את התשובה המתאימה

אם הניתוב הוא לפי וקטור-מרחק, אזי כאשר יש שינוי כלשהו במשקל של אחת מהקשתות בעלות משקל מינימלי, תמיד צומת אחד לפחות חייב לשלוח הודעת עדכון.

1. נכון
2. לא נכון

In distance-vector routing, when there is any change in the weight of one of the minimal-weighted edges, at least one node must always send an update message.

1. True
2. False

Question from an exam
Circle the correct answer

שאלה מבחינה (3 נקודות)
הקף בעיגול את התשובה המתאימה

אם הניתוב הוא לפי וקטור-מרחק, אזי כאשר יש שינוי כלשהו במשקל של אחת מהקשתות ולאחר השינוי קשת זו היא אחת מהקשתות בעלות המשקל המינימלי, תמיד צומת אחד לפחות חייב לשלוח הודעת עדכון.

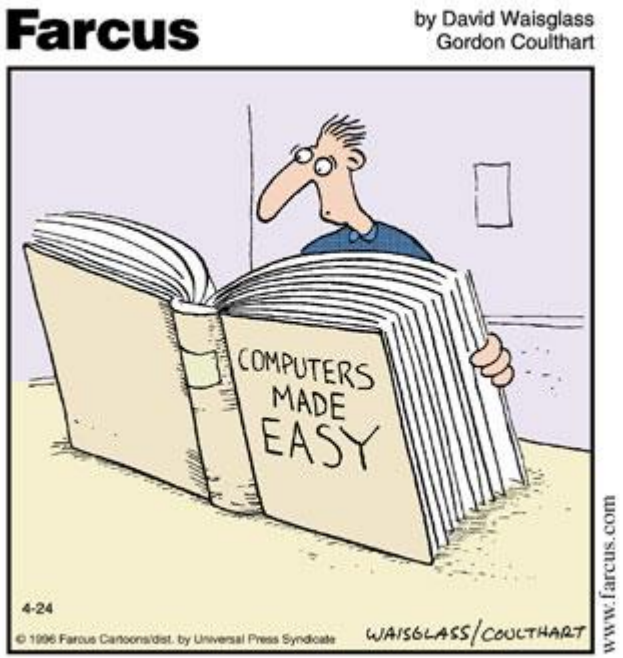
1. נכון
2. לא נכון

In distance-vector routing, when there is a change in the weight of one of the minimal-weighted edges, and its weight decreases, at least one node must always send an update message.

1. True
2. False

Today

- ✓ Transport layer
  - ✓ TCP – Connection Management
  - ✓ TCP – Flow Control
  - ✓ TCP – Congestion Control
  - ✓ TCP - Summary
- ✓ Network Layer
  - ✓ Recap – Review and Introduction
  - ✓ Routing – Link State (Dijkstra)
  - ✓ Routing – Distance Vector (Bellman Ford)
    - ✓ Basic algorithm
    - ✓ Cost change
    - ✓ Poisoned reverse
    - ✓ Real life solution
  - ✓ Comparing & Summary


Farcus
by David Waisglass
Gordon Coulthart
COMPUTERS
MADE
EASY
4-24
© 1996 Farcus Cartoons/dist. by Universal Press Syndicate
WAISGLASS/COULTHART
www.farcus.com